

CS3002E Compiler Design

Monsoon 2025

Lecture #23

Intermediate Representations

Department of Computer Science & Engineering
NIT Calicut

Intermediate Representations

The front end of a compiler generates an Intermediate Representation(IR) of the source program. Some of the common IRs are:

- Abstract Syntax Trees
- Three-address code
- DAG (Directed Acyclic Graph) for expressions
- Static Single-Assignment Form

Abstract Syntax Trees

Abstract Syntax Trees (or simply *syntax trees*)

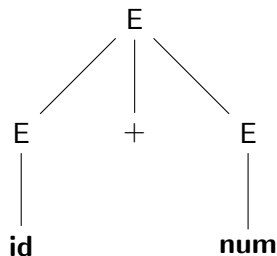
- Represents the hierarchical syntactic structure of the source program.
- Each construct is represented by an operator node with sub trees corresponding to the semantically meaningful components of the construct.
- Condensed form of *parse tree*¹, with the non terminal nodes either dropped or replaced by operators.

¹A parse tree is sometimes referred to as concrete syntax tree.

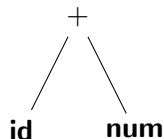
Parse Tree / Abstract Syntax Tree: Expressions

$$E \rightarrow E + E \mid E * E \mid id \mid num$$

Input string: $y + 10$



Parse Tree



Abstract Syntax Tree

SDD to construct syntax tree for expressions

Production	Semantic Rules
$E \rightarrow E_1 + E_2$	$E.node = CreateNode('+', E_1.node, E_2.node)$
$E \rightarrow E_1 * E_2$	$E.node = CreateNode('*', E_1.node, E_2.node)$
$E \rightarrow \mathbf{num}$	$E.node = CreateLeaf(\mathbf{num}, \mathbf{num.lexval})$
$E \rightarrow \mathbf{id}$	$E.node = CreateLeaf(\mathbf{id}, \mathbf{id.entry})$

Exercise

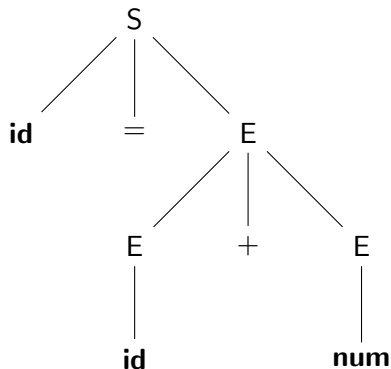
Complete the following SDD by writing Semantic Rules required to construct syntax tree for expressions. Draw the tree generated for $id_1 + id_2 * id_3$

Production	Semantic Rules
$E \rightarrow E_1 + T$	
$E \rightarrow T$	
$T \rightarrow T_1 * F$	
$T \rightarrow F$	
$F \rightarrow \mathbf{id}$	

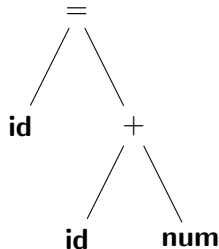
Parse Tree / Abstract Syntax Tree: Assignment Statement

$$S \rightarrow id = E$$
$$E \rightarrow E + E \mid E * E \mid id \mid num$$

Input string: $x = y + 10$



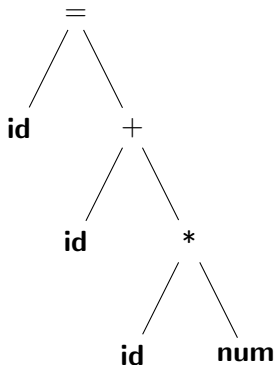
Parse Tree



Abstract Syntax Tree

Abstract Syntax Tree: Assignment Statement

Assignment: $x = \textit{sum} + i * 10$

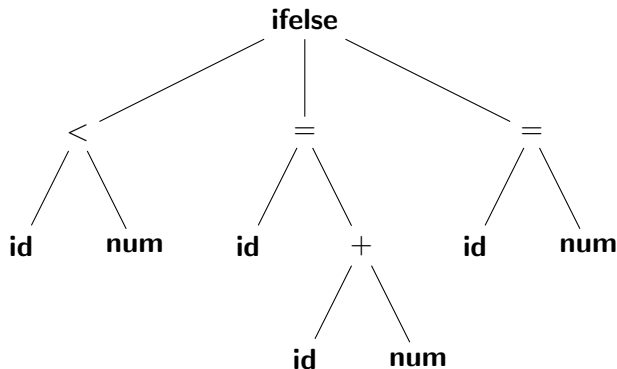


SDD to construct syntax tree for assignment

Production	Semantic Rules
$S \rightarrow id = E$	$S.node =$ $CreateNode('=', CreateLeaf(id, id.entry), E.node)$
$E \rightarrow E_1 + E_2$	$E.node = CreateNode('+', E_1.node, E_2.node)$
$E \rightarrow num$	$E.node = CreateLeaf(num, num.lexval)$
$E \rightarrow id$	$E.node = CreateLeaf(id, id.entry)$
...	...

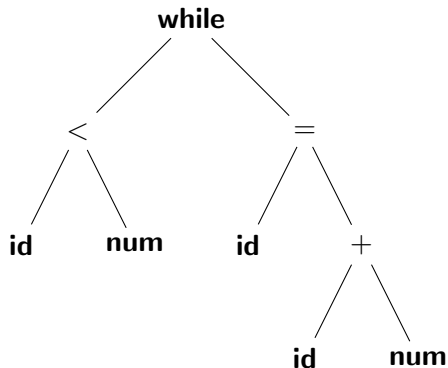
Abstract Syntax Tree: if-else statement

if-else statement: **if**($i < 10$) $i = i + 1$; **else** $i = 0$;



Abstract Syntax Tree: while statement

while loop: *while*($i < 100$) $i = i + 1$;



SDD to construct syntax tree

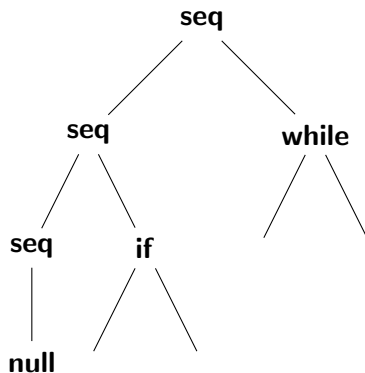
Production	Semantic Rules
$S \rightarrow \text{if } (B) S_1 \text{ else } S_2$	$S.\text{node} =$ $\text{CreateNode}(\text{ifelse}, B.\text{node}, S_1.\text{node}, S_2.\text{node})$
$S \rightarrow \text{while } (B) S_1$	$S.\text{node} =$ $\text{CreateNode}(\text{while}, B.\text{node}, S_1.\text{node})$
...	...

Note: The nonterminal B denotes a Boolean Expression

Abstract Syntax Tree: Statement sequence

$slist \rightarrow slist\ stmt \mid \epsilon$

e.g. **if**... followed by **while**...



operator **seq** for a sequence of statements, leaf **null** for an empty statement

Three-Address Code

- Instructions of the form $x=y \text{ op } z$ with three addresses - two for the operands and one for the result.
- At most one operator on the right side of an instruction.
- Temporary names (generated by the compiler) used for intermediate values.
- Instructions can have symbolic labels.
- Instructions for altering flow of control (conditional/unconditional *gotos*).

Three-Address Code: Addresses

An *address* in a 3-address instruction can be

- a name in the source program
- a constant
- a compiler-generated temporary

e.g. the instruction $t_1 = a + b$ contains 3 addresses: a and b are names in the source program, t_1 is a temporary.

Note: In the implementation, a name is replaced by a pointer to its *Symbol Table* entry.

Three-Address Code: example

The assignment $x = a + b * c$ is translated to the following sequence of 3-address instructions:

$$t_1 = b * c$$

$$t_2 = a + t_1$$

$$x = t_2$$

t_1 , t_2 are compiler generated temporaries

Common 3-address instructions

- Assignment instructions of the form:
 - $x = y \text{ op } z$ where op is a binary operator and x , y , and z are addresses
 - $x = op \ y$ where op is a unary operator and x , y , and z are addresses
- Copy instructions of the form $x = y$
- Unconditional jump `goto L` to transfer control to the instruction labelled L
- Conditional jump
 - `if x goto L` - jump to the instruction labelled L , if x is true
 - `ifFalse x goto L` - jump to the instruction labelled L , if x is false
 - `if x relop y goto L` - jump to the instruction labelled L , if $x \text{ relop } y$ is true ($relop$ denotes a relational operator)

3-address code: example

if ($a < b$) small = a else small = b ...

if $a < b$ goto L_1

goto L_2

L_1 : small=a

goto L_3

L_2 : small=b

L_3 : ...

Common 3-address instructions

- Procedure calls and returns: a procedure call $p(x_1, x_2, \dots x_n)$ is translated to the following 3-address instructions:

```
param  $x_1$   
param  $x_2$   
...  
param  $x_n$   
call  $p, n$ 
```

Function calls of the form $y = \text{call } p, n$

Returns of the form $\text{return } y$

- Indexed copy instructions of the form $x = y[i]$ and $x[i] = y$. ($x[i]$ refers to the location i memory units beyond x)
- Address and Pointer assignments of the form $x = \&y$, $x = *y$, and $*x = y$

3-address code: Representation

- Quadruples - each instruction as a record with four fields:
op - code for operator, *arg1*, *arg2* for operands and *result*.
- Triples - only three fields for each instruction. Result field is not part of the instruction. An instruction *i* can use the result of instruction *j* as operand, by keeping a reference to the position of instruction *j*.
- Indirect Triples - a list of pointers to triples.

3-address code: Quardruples

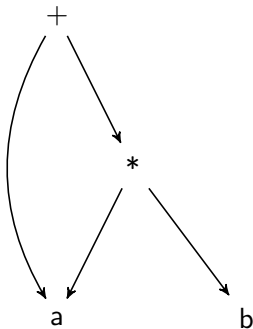
Each instruction as a record with four fields: *op* - code for operator, *arg₁*, *arg₂* for operands and *result*.

- Instructions with unary operators do not use *arg₂*
- *param* uses *arg₁* alone
- *goto* instruction keeps target label in *result*

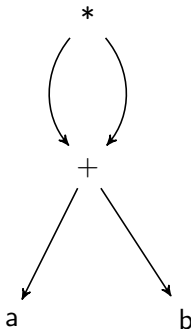
Directed Acyclic Graph(DAG) for Expressions

- Graph nodes corresponding to operands and operators.
- Common operand nodes (representing subexpressions) are not replicated.
- Can identify *common subexpressions*.

Example: DAG for $a + a * b$



Example: DAG for $(a + b) * (a + b)$



$a + b$ is a **common subexpression**. Code for evaluating $a + b$ needs to be generated only once. Avoids generating redundant code.

Static Single-Assignment (SSA) Form

- All assignments are to variables with distinct names.
- SSA facilitates certain code optimizations.
- Definitions of the same variable are changed to definitions of distinct variables by renaming of variables:

$$x = a + b$$

...

$$x = p + q$$

represented in SSA as

$$x_1 = a + b$$

...

$$x_2 = p + q$$

Static Single-Assignment (SSA) Form

SSA: Definition of the same variable in two different control-flow paths - use of ϕ functions:

```
if (...) x = 1 else x = 2;  
y = x;
```

represented in SSA as:

```
if (...) x1 = 1 else x2 = 2;  
x3 =  $\phi$ (x1, x2);  
y = x3;
```

The ϕ function returns the value of the argument corresponding to the control flow path taken to reach the statment containing it.

References

References:

- Aho A.V., Lam M.S., Sethi R., and Ullman J.D. Compilers: Principles, Techniques, and Tools (ALSU). Pearson Education, 2007.

Further reading:

- ALSU Chapter 2, Chapter 6